Step-by-Step Execution Guide — Deploy the AWS Cert Blueprint

How to build, verify, and tear down the full certification-practice environment

(VPC · security groups · RDS · EC2/ALB/Auto Scaling · SNS/SQS) with Terraform.

Note: See BLUEPRINT.md for the architecture diagram and design, and

Note: README.md for the data-engineering stack. This guide is the

Note: runbook for actually deploying the blueprint modules.

0. Prerequisites (once)

#	Check	Command
1	Terraform installed	terraform -version (>= 1.5)
2	AWS CLI installed	aws --version
3	Credentials work	aws sts get-caller-identity → shows your account
4	Region set	aws configure set region us-west-2
5	Billing alarm set	Console → Billing → Budgets → alert at e.g. $5

1. Understand the toggles (why nothing built the first time)

Every blueprint module is OFF by default so you never create paid resources by

accident. You turn them on with -var flags (or a .tfvars file).

Toggle	Creates	Paid?
enable_vpc	VPC, subnets, IGW, NAT, route tables, endpoints	NAT is paid
enable_security_groups	the 5 tiered SGs (needs VPC)	free
enable_rds	RDS DB in private subnets (needs VPC + SGs)	free tier if single-AZ
enable_ec2_asg	EC2 + ALB + Auto Scaling (needs VPC + SGs)	ALB is paid
enable_messaging	SNS + SQS + DLQ	free

Note: ⚠️ Common mistake: running terraform apply *without* the flags only builds

Note: the default-on data-engineering stack — the VPC/RDS/ALB won't appear, and

Note: security_group_ids will show null. Always pass the flags (or a tfvars file)

Note: to build the blueprint.

Recommended: use a tfvars file (so you don't retype 5 flags)

Create dev.tfvars (already gitignored):

```
enable_vpc             = true
enable_security_groups = true
enable_rds             = true
enable_ec2_asg         = true
enable_messaging       = true

# lock SSH to your IP (find it: curl ifconfig.me)
my_ip_cidr = "YOUR_IP/32"
```

Then every command below is just ... -var-file=dev.tfvars.

2. Initialize (required after adding/changing modules)

```
cd terraform-aws-practice
terraform init
```

init downloads providers and registers the local modules. It creates nothing

in AWS. Run it any time you add a module — skipping it causes the

Error: Module not installed message.

3. Format & validate (free, catches errors early)

```
terraform fmt -recursive
terraform validate
```

Expect: Success! The configuration is valid.

4. Plan — preview what will be built (free, creates nothing)

```
terraform plan -var-file=dev.tfvars
```

Read the bottom line, e.g. Plan: ~40 to add, 0 to change, 0 to destroy. Skim the

list — you should see aws_vpc, aws_subnet (×4), aws_nat_gateway,

aws_security_group (×5), aws_db_instance, aws_lb, aws_autoscaling_group,

aws_sns_topic, aws_sqs_queue (×2), etc.

5. Apply — build the environment

```
terraform apply -var-file=dev.tfvars
```

Type yes. This takes ~10–15 minutes — RDS alone is ~8–10 min (normal, not

stuck). When done you'll see:

```
Apply complete! Resources: N added, 0 changed, 0 destroyed.

Outputs:
alb_dns_name            = "practice-alb-xxxx.us-west-2.elb.amazonaws.com"
rds_endpoint            = "practice-rds.xxxx.us-west-2.rds.amazonaws.com:3306"
sns_topic_arn           = "arn:aws:sns:us-west-2:...:practice-topic"
sqs_queue_url           = "https://sqs.us-west-2.amazonaws.com/.../practice-queue"
vpc_id                  = "vpc-xxxx"
vpc_public_subnet_ids   = ["subnet-...","subnet-..."]
vpc_private_subnet_ids  = ["subnet-...","subnet-..."]
security_group_ids      = { alb = "sg-...", app = "sg-...", ... }
```

Note: If security_group_ids shows null and there's no vpc_id, you forgot the

Note: -var-file=dev.tfvars — re-run apply with it.

6. Verify it works

```
terraform output          # collect alb_dns_name, rds_endpoint, sqs_queue_url
```

A. Load balancer → private EC2 (the request path):

```
# wait ~2 min for instances to pass ALB health checks, then:
curl http://$(terraform output -raw alb_dns_name)
# expect: <h1>practice app server - <hostname></h1>
```

Or open http://<alb_dns_name> in a browser.

B. Auto Scaling Group is running instances:

```
aws autoscaling describe-auto-scaling-groups --region us-west-2 \
  --query "AutoScalingGroups[?
contains(AutoScalingGroupName,'practice')].Instances[].
[InstanceId,LifecycleState,HealthStatus]" \
  --output table
```

C. RDS is up and private:

```
aws rds describe-db-instances --region us-west-2 \
  --query "DBInstances[?DBInstanceIdentifier=='practice-rds'].
[DBInstanceStatus,PubliclyAccessible,MultiAZ]" \
  --output table
# expect: available | False | (False for single-AZ / True for HA)
```

PubliclyAccessible = False proves the DB is not exposed — a key exam point.

D. Messaging chain (SNS → SQS):

```
aws sns list-subscriptions --region us-west-2 \
  --query "Subscriptions[?contains(TopicArn,'practice')].[Protocol,Endpoint]" --
output table
# publish a test message and read it off the queue:
aws sns publish --region us-west-2 \
  --topic-arn $(terraform output -raw sns_topic_arn) \
  --message '{"hello":"world"}'
aws sqs receive-message --region us-west-2 \
  --queue-url $(terraform output -raw sqs_queue_url)
```

7. Tear it all down (stop charges)

```
terraform destroy -var-file=dev.tfvars
```

Type yes. This removes the VPC, NAT, RDS, ALB, ASG, SGs, and messaging. Verify

nothing paid is left:

```
aws ec2 describe-nat-gateways --region us-west-2 --query "NatGateways[?
State=='available']" --output text   # should be empty
aws rds describe-db-instances --region us-west-2 --query
"DBInstances[].DBInstanceIdentifier" --output text  # should be empty
aws elbv2 describe-load-balancers --region us-west-2 --query
"LoadBalancers[].LoadBalancerName" --output text # should be empty
```

Note: The data-engineering stack (S3/DynamoDB/Lambda/Glue) is separate and stays unless

Note: you destroy without the flags too. To remove *everything*, run

Note: terraform destroy -var-file=dev.tfvars — the flags only add resources, so a

Note: plain destroy plus the flags removes both stacks.

8. Troubleshooting (real issues & fixes)

Symptom	Cause	Fix
Error: Module not installed	added a module, didn't init	terraform init
security_group_ids = null, no vpc_id/ alb_dns_name	forgot the toggle flags	re-run with -var-file=dev.tfvars
InvalidClientTokenId (403)	bad/expired credentials	new access key + aws configure; verify aws sts get-caller-identity
ALB shows 502/503	instances not healthy yet	wait ~2 min for health checks; check the target group health
RDS apply "takes forever"	RDS provisioning is slow	normal — ~8–10 min; wait
1 destroyed on a plain apply	latest AMI changed, EC2 replaced	expected with most_recent = true

9. Cost summary

Resource	Cost	Notes
NAT Gateway	~$0.045/hr + data	paid — the main cost
Application Load Balancer	~$0.0225/hr + LCU	paid
RDS db.t3.micro	free tier (single-AZ)	Multi-AZ is not free
EC2 t3.micro ×2	free tier	750 hrs/mo combined
S3, DynamoDB, Lambda, SNS, SQS	free tier	generous

Rough total if left running: ~$1–2/day. Spin up → verify → **destroy the same

day**. Always keep a Billing Budget alarm active.

The build order (recap)

`Identity → Network (VPC F-L) → Firewalls (SGs, M) → Data (RDS, P) →

Compute (EC2/ALB, Q-U) → Messaging (V-X) → Serverless → Monitoring.`

Terraform figures out the exact dependency order from the module references — this

is why one apply builds it all correctly.